Disjoint Set Union (Union-Find) and Connected Components

Design and Analysis of Algorithms — Lab Report (medium and hard problems)

#	Problem	Time	Space
Q1	Online Graph Bipartiteness Testing using DSU with Parity	`O(V + E α(V))`	`O(V)`
Q2	Dynamic Connected Components in a Grid (Number of Islands II)	`O(R·C + k α(R·C))`	`O(R·C)`
Q3	Maximum Weight Component with Size Constraint	`O(E α(V) + V log V)`	`O(V)`
Q4	Reverse Time-Step DSU: Maximum Component Sum under Vertex Deletions	`O(V + E α(V))`	`O(V + E)`
Q5	Lexicographically Smallest Permutation using Allowed Swaps	`O(n + m α(n))`	`O(n)`

Each problem section contains: problem statement and modelling assumptions, input representation, possible approaches, the basic idea, the step-by-step algorithm, the complete C program, important functions and variables, sample input/output (produced by actually running the program), a dry run, complexity analysis with derivation, a measured running-time graph, advantages and limitations, viva questions and a short teacher explanation.

Validation. All five programs were compiled with `gcc -Wall -Wextra` without warnings and checked against independent brute-force implementations on 500 random inputs each (all outputs matched). α(n) denotes the inverse Ackermann function, which is at most 4 for any practical n.

Q1. Online Graph Bipartiteness Testing using DSU with Parity

1. Problem Title

Online Graph Bipartiteness Testing using DSU with Parity

2. Problem Statement

A graph is bipartite **if and only if** it contains no odd-length cycle. We are given an undirected graph with `V` vertices. Edges arrive **online**, one at a time. For every arriving edge `(u, v)`:

- if adding `(u, v)` does **not** create an odd cycle, the edge is added;
- otherwise the program reports that the edge violates bipartiteness and **skips** it.

Hence the graph that is actually maintained is always bipartite. The program must validate the algorithm in C and give a complexity analysis.

Modelling assumptions

- Vertices are numbered `1..V`.
- A self-loop `(u, u)` is an odd cycle of length 1, so it is always rejected.
- A repeated edge that already exists is harmless: its endpoints already have different colours, so it is accepted without changing anything.
- Edges are processed strictly in arrival order and are never stored: the program is truly online.

3. Input Representation

Because the edges arrive online, an adjacency matrix or adjacency list would be wasteful and is not needed. The only information we must remember about the graph is **which vertices are connected and whether two connected vertices have the same or different colour**. We therefore represent the graph implicitly by three arrays of size `V + 1`:

Array	Meaning
`parent[x]`	parent of `x` in the DSU forest (`parent[x] == x` for a root)
`parity[x]`	colour difference between `x` and `parent[x]` (0 = same colour, 1 = different colour)
`rnk[x]`	rank of the tree rooted at `x` (used for union by rank)

The edges are read from standard input as plain pairs `u v`:

```
V
E
u1 v1
u2 v2
...
uE vE
```

4. Possible Approaches

Approach 1: Re-run BFS/DFS 2-colouring after every edge

Store the graph, add the new edge and run a full BFS colouring. This is correct but costs `O(V + E)` for each arriving edge, so `O(E · (V + E))` in total. It is too slow for a large online stream.

Approach 2: Doubled-vertex DSU

Create two nodes `v` and `v'` for each vertex. For edge `(u, v)` unite `u` with `v'` and `u'` with `v`. The edge creates an odd cycle exactly when `u` and `v` are already in the same set. This works and uses `2V` nodes.

Approach 3: DSU with parity (used here)

Keep one node per vertex and store, along every DSU link, whether the two ends have the same or different colour. This has the same complexity as Approach 2, uses only `V` nodes and directly produces a valid 2-colouring at the end.

5. Basic Idea

Every connected component of a bipartite graph has an essentially unique 2-colouring (up to swapping the two colours). The DSU stores that colouring **relative to the root** of each component:

```
colour(x) = XOR of parity values on the path from x up to its root  (colour(root) = 0)
```

When edge `(u, v)` arrives, let `ru = find(u)` and `rv = find(v)`:

- `ru == rv` (same component): the colours of `u` and `v` are already forced. If `colour(u) == colour(v)` the new edge closes an **odd cycle**, so reject it. Otherwise it closes an even cycle and is accepted (nothing changes).
- `ru != rv` (different components): no cycle can be formed. Merge the two components and set the parity of the attached root to `parity[u] ^ parity[v] ^ 1`, which forces `u` and `v` to get different colours.

6. Step-by-Step Algorithm

1 Read `V` and `E`. Set `parent[i] = i`, `parity[i] = 0`, `rnk[i] = 0` for all vertices; `components = V`.

2 For each arriving edge `(u, v)`: if `u == v`, reject it.

3 Compute `ru = find(u)` and `rv = find(v)`. `find` also leaves `parity[u]` and `parity[v]` equal to the colour of `u` and `v` relative to their roots.

4 If `ru == rv`: reject when `parity[u] == parity[v]`, accept otherwise.

5 If `ru != rv`: attach the root of smaller rank below the other root, set its parity to `parity[u] ^ parity[v] ^ 1`, and decrease `components`.

6 Print ACCEPTED or REJECTED for the edge.

7 At the end print the number of accepted/rejected edges, the number of components and a valid 2-colouring.

7. C Program

The complete program is in `Q1.c`. Prompts are written to `stderr` and results to `stdout`.

```c
#include <stdio.h>
#include <stdlib.h>

/*
   Online bipartiteness testing using DSU with parity.

   parent[x] : parent of x in the DSU forest
   parity[x] : colour difference between x and parent[x] (0 = same, 1 = different)
   rnk[x]    : rank (upper bound on height) used for union by rank
*/

int V, E;
int *parent, *parity, *rnk;
int components;

/* Find root of x. After the call, parent[x] is the root (or x is the root)
   and parity[x] is the colour difference between x and its root.
   Iterative, with path compression, so there is no recursion depth problem. */
int find(int x)
{
    int root = x, p = 0, cur, cp, next, np;

    /* pass 1: find the root and the parity of x relative to the root */
    while (parent[root] != root)
    {
        p ^= parity[root];
        root = parent[root];
    }

    /* pass 2: path compression, keeping every parity correct */
    cur = x;
    cp = p;
    while (parent[cur] != cur)
    {
        next = parent[cur];
        np = cp ^ parity[cur];   /* parity of next relative to the root */
        parent[cur] = root;
        parity[cur] = cp;
        cur = next;
        cp = np;
    }
    return root;
}
```

```c
/* Returns 1 if edge (u,v) is accepted, 0 if it would create an odd cycle */
int addEdge(int u, int v)
{
    int ru, rv;

    if (u == v)                 /* self-loop = odd cycle of length 1 */
        return 0;

    ru = find(u);
    rv = find(v);

    if (ru == rv)
    {
        /* same component: u and v must have different colours */
        if (parity[u] == parity[v])
            return 0;           /* odd cycle would be formed */
        return 1;               /* even cycle, graph stays bipartite */
    }

    /* different components: merge them so that u and v get different colours */
    if (rnk[ru] < rnk[rv])
    {
        parent[ru] = rv;
        parity[ru] = parity[u] ^ parity[v] ^ 1;
    }
    else
    {
        parent[rv] = ru;
        parity[rv] = parity[u] ^ parity[v] ^ 1;
        if (rnk[ru] == rnk[rv])
            rnk[ru]++;
    }
    components--;
    return 1;
}

int main()
{
    int i, u, v, accepted = 0, rejected = 0;

    fprintf(stderr, "Enter number of vertices V: ");
    if (scanf("%d", &V) != 1 || V <= 0)
    {
        printf("Number of vertices must be positive.\n");
        return 0;
    }
    fprintf(stderr, "Enter number of edges E: ");
    if (scanf("%d", &E) != 1 || E < 0)
    {
        printf("Number of edges must not be negative.\n");
        return 0;
    }

    parent = (int *)malloc((V + 1) * sizeof(int));
    parity = (int *)calloc(V + 1, sizeof(int));
    rnk    = (int *)calloc(V + 1, sizeof(int));
    if (!parent || !parity || !rnk)
    {
        printf("Memory allocation failed.\n");
        return 0;
    }
    for (i = 1; i <= V; i++)
        parent[i] = i;
    components = V;

    fprintf(stderr, "Enter the %d edges (u v), vertices are 1..%d:\n", E, V);
    for (i = 1; i <= E; i++)
    {
        fprintf(stderr, "Edge %d: ", i);
        if (scanf("%d %d", &u, &v) != 2 || u < 1 || u > V || v < 1 || v > V)
        {
            printf("Invalid edge. Vertices must be between 1 and %d.\n", V);
            return 0;
        }

        if (addEdge(u, v))
        {
            accepted++;
            printf("Edge %d (%d, %d): ACCEPTED\n", i, u, v);
        }
        else
        {
            rejected++;
            printf("Edge %d (%d, %d): REJECTED (odd cycle)\n", i, u, v);
        }
    }
```

```
    printf("\nAccepted edges = %d, rejected edges = %d\n", accepted, rejected);
    printf("Connected components = %d\n", components);
    printf("The accepted graph is bipartite.\n");

    /* colour of a vertex = parity relative to its root; a root gets colour 0 */
    printf("\nA valid 2-colouring:\n");
    for (i = 1; i <= V; i++)
    {
        find(i);
        printf("Vertex %d -> side %d\n", i, (i == parent[i]) ? 0 : parity[i]);
    }

    free(parent); free(parity); free(rnk);
    return 0;
}
```

8. Important Functions

`find(int x)`

Purpose: returns the root of `x` and compresses the path while keeping every parity value correct.
How it works: pass 1 walks up to the root and XORs the parities seen on the way, giving the parity of `x` relative to the root. Pass 2 walks the same path again and makes every node point directly to the root, storing for each node its own parity relative to the root. It is iterative, so no recursion-depth problem occurs for large `V`.

`addEdge(int u, int v)`

Purpose: processes one arriving edge.
Returns: `1` if the edge is accepted, `0` if it is rejected (self-loop or odd cycle). Implements the case analysis described in *Basic Idea*, with union by rank.

`main()`

Reads the input, allocates the arrays, calls `addEdge` for every edge and prints the results and the final colouring. Prompts are written to `stderr` and results to `stdout`, so the output can be redirected cleanly to a file.

9. Important Variables

Variable	Meaning
`V`, `E`	Number of vertices and of arriving edges
`parent[]`	DSU parent of each vertex
`parity[]`	Colour difference between a vertex and its parent
`rnk[]`	Rank used for union by rank
`components`	Current number of connected components
`accepted`, `rejected`	Counters of accepted / skipped edges
`ru`, `rv`	Roots of the two endpoints of the current edge

10. Sample Input

Type the values below (or redirect them from a file). The prompts are printed on the screen while you type; only the raw input is shown here.

```
5
8
1 2
2 3
3 4
4 1
1 3
4 5
5 2
5 1
```

11. Sample Output

```
Edge 1 (1, 2): ACCEPTED
Edge 2 (2, 3): ACCEPTED
Edge 3 (3, 4): ACCEPTED
Edge 4 (4, 1): ACCEPTED
```

```
Edge 5 (1, 3): REJECTED (odd cycle)
Edge 6 (4, 5): ACCEPTED
Edge 7 (5, 2): ACCEPTED
Edge 8 (5, 1): REJECTED (odd cycle)

Accepted edges = 6, rejected edges = 2
Connected components = 1
The accepted graph is bipartite.

A valid 2-colouring:
Vertex 1 -> side 0
Vertex 2 -> side 1
Vertex 3 -> side 0
Vertex 4 -> side 1
Vertex 5 -> side 0
```

This output was produced by compiling and running the program on the sample input above.

12. Dry Run

Take the sample input (`V = 5`). Initially every vertex is its own root with `parity = 0`.

Edge	Roots (ru, rv)	Action	Result
1: (1,2)	1, 2	different roots; attach 2 under 1, `parity[2] = 0^0^1 = 1`	ACCEPTED
2: (2,3)	1, 3	`parity[2] = 1`, `parity[3] = 0`; attach 3 under 1, `parity[3] = 1^0^1 = 0`	ACCEPTED
3: (3,4)	1, 4	attach 4 under 1, `parity[4] = 0^0^1 = 1`	ACCEPTED
4: (4,1)	1, 1	same root; `parity[4] = 1`, `parity[1] = 0` differ (even cycle 1-2-3-4)	ACCEPTED
5: (1,3)	1, 1	same root; `parity[1] = 0`, `parity[3] = 0` equal (triangle 1-2-3)	REJECTED
6: (4,5)	1, 5	attach 5 under 1, `parity[5] = 1^0^1 = 0`	ACCEPTED
7: (5,2)	1, 1	`parity[5] = 0`, `parity[2] = 1` differ (even cycle 2-3-4-5)	ACCEPTED
8: (5,1)	1, 1	`parity[5] = 0`, `parity[1] = 0` equal (odd cycle 1-4-5)	REJECTED

Final colours (parity relative to root 1): vertices 1, 3, 5 get side 0 and vertices 2, 4 get side 1. Every accepted edge joins a vertex of side 0 to a vertex of side 1, confirming that the maintained graph is bipartite.

13. Complexity Analysis

With **union by rank** and **path compression**, a sequence of `m` `find` operations on `V` elements takes `O(m · α(V))` time (Tarjan), where `α` is the inverse Ackermann function, which is at most 4 for any practical `V`.

Quantity	Cost
Initialisation	`O(V)`
One edge (two `find` calls + one link)	`O(α(V))` amortised
All `E` edges	`O(E · α(V))`
Final 2-colouring output	`O(V · α(V))`
Total time	`O(V + E · α(V))`, practically linear
Space	`O(V)`: three arrays; the edges are not stored

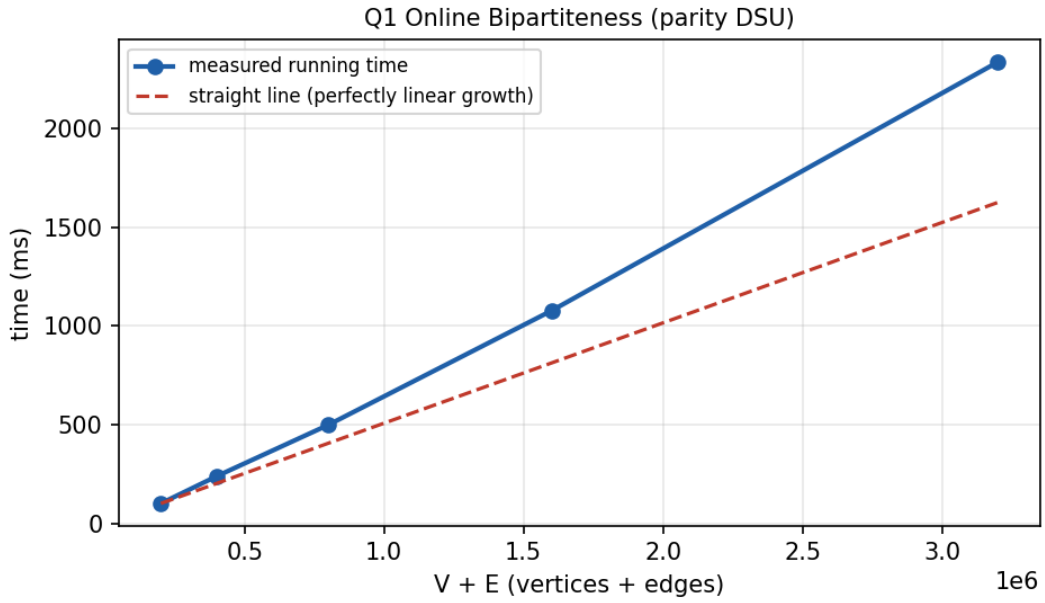
Best, average and worst cases are all `O(V + E · α(V))`, because every edge needs exactly two `find` calls regardless of the answer.

How it was derived

Each edge does a constant amount of work apart from two `find` calls and at most one link. The `find` cost is amortised `O(α(V))` because of the two heuristics. Summing over `E` edges gives `O(E α(V))`, and the `O(V)` initialisation is added once. Compared with Approach 1 (`O(E(V+E))`) this is a huge improvement.

14. Complexity Graph

The graph below shows the **measured running time** of the program (best of 3 runs, `gcc -O2`) for increasing input sizes (up to 1.6 million vertices or operations, with as many edges), together with a perfectly linear reference line. The measured times include reading the input with `scanf`, which is itself linear. The curve stays close to the straight line, which agrees with the near-linear bound.



Q1: measured running time against input size

15. Advantages

1. Truly online: each edge is handled the moment it arrives, with no stored graph.
2. Near-linear time and only `O(V)` extra memory.
3. Gives a valid 2-colouring of the maintained graph as a by-product.
4. Iterative `find`, so there is no stack overflow for large inputs.

16. Limitations

1. Edges can only be added, never deleted (DSU cannot split sets).
2. Rejected edges are skipped, so the program cannot later tell which edges were rejected unless they are recorded separately.
3. The colouring is only one of the possible valid colourings (the two sides of every component can be swapped).

17. Viva Preparation

Q1. What is the main idea?

Use DSU where every link also stores whether the two ends have the same or different colour. An edge inside one component is bad exactly when both ends already have the same colour.

Q2. Why does 'no odd cycle' mean bipartite?

A graph is bipartite iff it is 2-colourable iff it contains no odd-length cycle.

Q3. What does `parity[x]` store?

The colour difference between `x` and `parent[x]`: 0 if same, 1 if different.

Q4. How is the colour of a vertex obtained?

XOR of the parity values along the path to the root; the root has colour 0.

Q5. Why `parity[u] ^ parity[v] ^ 1` while merging?

After the merge `u` and `v` must have different colours. The parity of the attached root is chosen so that the XOR of the two paths plus this link equals 1.

Q6. Why is a self-loop rejected?
It is a cycle of length 1, which is odd.

Q7. What if the edge connects two vertices in the same component with different colours?
It closes an even cycle, so it is accepted and nothing needs to be updated.

Q8. What is the time complexity?
`O(V + E α(V))`, practically linear, with `O(V)` space.

Q9. Why union by rank and path compression?
Together they make `find` take amortised `O(α(V))` time.

Q10. Why is `find` iterative?
A recursive `find` could overflow the stack on a long chain when `V` is large.

Q11. Why not run BFS after each edge?
It costs `O(V+E)` per edge, i.e. `O(E(V+E))` overall.

Q12. Can the algorithm support edge deletion?
No. Deletion needs offline techniques such as divide and conquer on time with a rollback DSU.

18. 1–2 Minute Teacher Explanation

> *"A graph is bipartite exactly when it has no odd cycle, so I only have to detect whether a new edge would close an odd cycle. I use a disjoint set union where every node also stores a parity bit: whether it has the same or the opposite colour as its parent. The colour of a vertex relative to its root is the XOR of the parities on its path.*
>
> *When an edge (u, v) arrives I find both roots. If they are different, the edge cannot form a cycle, so I merge the components and set the parity of the attached root so that u and v receive opposite colours. If the roots are equal, I compare the colours of u and v: equal colours mean the edge would close an odd cycle, so I reject it; different colours mean an even cycle, so I accept it.*
>
> *With union by rank and path compression each edge costs amortised O(α(V)), so the whole stream takes O(V + E α(V)) time and O(V) space. At the end the parities directly give a valid 2-colouring.*"

19. How to Compile and Run

Save the program as `Q1.c`.

GCC (Linux / macOS)

```
gcc Q1.c -o Q1
./Q1
```

Windows

```
gcc Q1.c -o Q1.exe
Q1.exe
```

Using a file as input

```
./Q1 < input.txt > output.txt
```

20. Files in This Lab

1. `Q1.c` — C source code
2. `README.md` — complete explanation and lab documentation
3. `Q1_complexity.png` — measured running-time graph

21. Conclusion

Online bipartiteness testing reduces to detecting whether a new edge closes an odd cycle. Storing a parity bit on every DSU link lets the program answer this by comparing two XOR values, so each edge costs amortised `O(α(V))`. The total complexity is `O(V + E α(V))` time and `O(V)` space.

Q2. Dynamic Connected Components in a Grid (Number of Islands II)

1. Problem Title

Dynamic Connected Components in a Grid (Number of Islands II)

2. Problem Statement

An `R × C` grid is initially completely water. Land cells are added one by one at positions `(r1, c1), (r2, c2), …, (rk, ck)`. After **every** operation the program must print the current number of 4-directionally connected land components (islands).

Modelling assumptions

- Rows and columns are numbered from `0` (`0 ≤ r < R`, `0 ≤ c < C`), as in the standard Number of Islands II problem.
- Connectivity is 4-directional: up, down, left, right (diagonals do not connect).
- If a cell that is already land is added again, nothing changes and the island count is printed again.
- A position outside the grid is reported and ignored.

3. Input Representation

The grid is never stored as a 2-D matrix of characters. Every cell `(r, c)` is **flattened** into a single number:

```
id = r * C + c          (and back:  r = id / C,  c = id % C)
```

A single integer array `parent[0 .. R*C-1]` represents the whole grid and the DSU at the same time:

- `parent[id] = -1` means the cell is **water**;
- otherwise the cell is **land** and `parent[id]` is its DSU parent (`parent[id] == id` for a root).
- `sz[id]` stores the size of the tree whose root is `id` (union by size).

The input is one line `R C K` followed by `K` lines `r c`:

```
R C K
r1 c1
r2 c2
...
rK cK
```

4. Possible Approaches

Approach 1: BFS/DFS after every operation

Count the islands from scratch after each new land cell. It costs `O(R·C)` per operation, so `O(k · R · C)` overall: far too slow for large grids.

Approach 2: DSU with coordinate flattening (used here)

Adding a land cell can only create a new island and merge it with at most four neighbouring islands. DSU handles exactly these operations (make-set and union) in almost constant time.

5. Basic Idea

Maintain a counter `islands`. When a cell `(r, c)` becomes land:

1. If it is already land, do nothing.
2. Make it a new DSU set and do `islands++` (a new island is born).
3. For each of the four neighbours that is land, `unite` the two cells. Every time two **different** sets are merged, do `islands--`.

The counter is therefore always correct and no recounting is needed.

6. Step-by-Step Algorithm

1. Read `R`, `C`, `K`. Allocate `parent` and `sz` of size `R·C`; set every `parent[i] = -1`; `islands = 0`.

2 For each operation read `(r, c)`; if it is outside the grid, report and skip.

3 Let `id = r*C + c`. If `parent[id] != -1`, the cell is already land: print the unchanged count.

4 Otherwise set `parent[id] = id`, `sz[id] = 1` and `islands++`.

5 For each of the four neighbours inside the grid that is land: if `unite(id, neighbour)` merged two different sets, do `islands--`.

6 Print the current `islands`.

7 After all operations print the final grid (if it is small enough).

7. C Program

The complete program is in `Q2.c`. Prompts are written to `stderr` and results to `stdout`.

```
#include <stdio.h>
#include <stdlib.h>

/*
   Dynamic connected components in a grid (Number of Islands II).
   Cell (r,c) is flattened to id = r*C + c.
   parent[id] = -1  -> the cell is water
   otherwise it is a land cell inside a DSU tree.
*/

int R, C, K;
int *parent, *sz;
int islands = 0;

int find(int x)
{
    while (parent[x] != x)
    {
        parent[x] = parent[parent[x]];   /* path halving */
        x = parent[x];
    }
    return x;
}

/* Merge the sets of a and b; returns 1 if they were different */
int unite(int a, int b)
{
    int ra = find(a), rb = find(b);
    if (ra == rb)
        return 0;
    if (sz[ra] < sz[rb])
    {
        int t = ra; ra = rb; rb = t;
    }
    parent[rb] = ra;
    sz[ra] += sz[rb];
    return 1;
}

/* Turn cell (r,c) into land. Returns 0 if it was already land. */
int addLand(int r, int c)
{
    int id = r * C + c, d;
    int dr[4] = {-1, 1, 0, 0};
    int dc[4] = {0, 0, -1, 1};

    if (parent[id] != -1)
        return 0;

    parent[id] = id;
    sz[id] = 1;
    islands++;                           /* a new island is born */

    for (d = 0; d < 4; d++)
    {
        int nr = r + dr[d], nc = c + dc[d];
        if (nr < 0 || nr >= R || nc < 0 || nc >= C)
            continue;
        if (parent[nr * C + nc] == -1)
            continue;                    /* neighbour is water */
        if (unite(id, nr * C + nc))
            islands--;                   /* two islands became one */
    }
    return 1;
}

int main()
{
```

```
    int i, r, c;

    fprintf(stderr, "Enter rows R, columns C and number of operations K: ");
    if (scanf("%d %d %d", &R, &C, &K) != 3 || R <= 0 || C <= 0 || K < 0)
    {
        printf("R and C must be positive and K must not be negative.\n");
        return 0;
    }

    parent = (int *)malloc((size_t)R * C * sizeof(int));
    sz     = (int *)calloc((size_t)R * C, sizeof(int));
    if (!parent || !sz)
    {
        printf("Memory allocation failed.\n");
        return 0;
    }
    for (i = 0; i < R * C; i++)
        parent[i] = -1;                 /* whole grid is water */

    fprintf(stderr, "Enter K positions (row col), 0-indexed:\n");
    for (i = 1; i <= K; i++)
    {
        fprintf(stderr, "Operation %d: ", i);
        if (scanf("%d %d", &r, &c) != 2)
        {
            printf("Invalid input.\n");
            return 0;
        }
        if (r < 0 || r >= R || c < 0 || c >= C)
        {
            printf("Operation %d (%d, %d): outside grid, ignored. "
                   "islands = %d\n", i, r, c, islands);
            continue;
        }
        if (!addLand(r, c))
            printf("Operation %d (%d, %d): already land. islands = %d\n",
                   i, r, c, islands);
        else
            printf("Operation %d (%d, %d): islands = %d\n", i, r, c, islands);
    }

    if (R <= 30 && C <= 30)
    {
        printf("\nFinal grid (# = land, . = water):\n");
        for (r = 0; r < R; r++)
        {
            for (c = 0; c < C; c++)
                putchar(parent[r * C + c] == -1 ? '.' : '#');
            putchar('\n');
        }
    }

    free(parent); free(sz);
    return 0;
}
```

8. Important Functions

`find(int x)`

Returns the root of `x`, using **path halving** (each visited node is pointed to its grandparent), which gives the same amortised bound as full path compression and is iterative.

`unite(int a, int b)`

Merges the sets of `a` and `b` by size. Returns `1` if they were in different sets (a merge happened) and `0` otherwise. The return value is what lets the caller decrement `islands` correctly.

`addLand(int r, int c)`

Turns one cell into land, creates its singleton set, increments `islands` and merges with the up/down/left/right land neighbours. Returns `0` if the cell was already land.

9. Important Variables

Variable	Meaning
`R`, `C`, `K`	Rows, columns, number of operations
`parent[]`	`-1` = water, otherwise DSU parent of a land cell
`sz[]`	Size of the tree of each root (union by size)

Variable	Meaning
`islands`	Current number of islands
`id`	Flattened index `r*C + c` of a cell
`dr[]`, `dc[]`	Row/column offsets of the 4 neighbours

10. Sample Input

Type the values below (or redirect them from a file). The prompts are printed on the screen while you type; only the raw input is shown here.

```
3 3 6
0 0
0 1
1 2
2 1
1 1
0 0
```

11. Sample Output

```
Operation 1 (0, 0): islands = 1
Operation 2 (0, 1): islands = 1
Operation 3 (1, 2): islands = 2
Operation 4 (2, 1): islands = 3
Operation 5 (1, 1): islands = 1
Operation 6 (0, 0): already land. islands = 1

Final grid (# = land, . = water):
##.
.##
.#.
```

This output was produced by compiling and running the program on the sample input above.

12. Dry Run

Grid `3 × 3`, `id = r*3 + c`. Operations: `(0,0) (0,1) (1,2) (2,1) (1,1) (0,0)`.

Op	Cell (id)	Neighbouring land	Effect on `islands`	Count
1	(0,0) id 0	none	+1	1
2	(0,1) id 1	(0,0): merge	+1, −1	1
3	(1,2) id 5	none	+1	2
4	(2,1) id 7	none	+1	3
5	(1,1) id 4	up (0,1), down (2,1), right (1,2): three merges	+1, −1, −1, −1	1
6	(0,0) id 0	already land	none	1

The final grid is:

```
##.
.##
.#.
```

13. Complexity Analysis

Let `N = R·C` and let `k` be the number of operations.

Quantity	Cost
Initialisation of `parent` (all water)	`O(N)`
One land addition: one make-set + at most 4 `unite` calls	`O(α(N))` amortised
All `k` operations	`O(k · α(N))`
Total time	`O(R·C + k · α(R·C))`
Space	`O(R·C)` for `parent` and `sz`

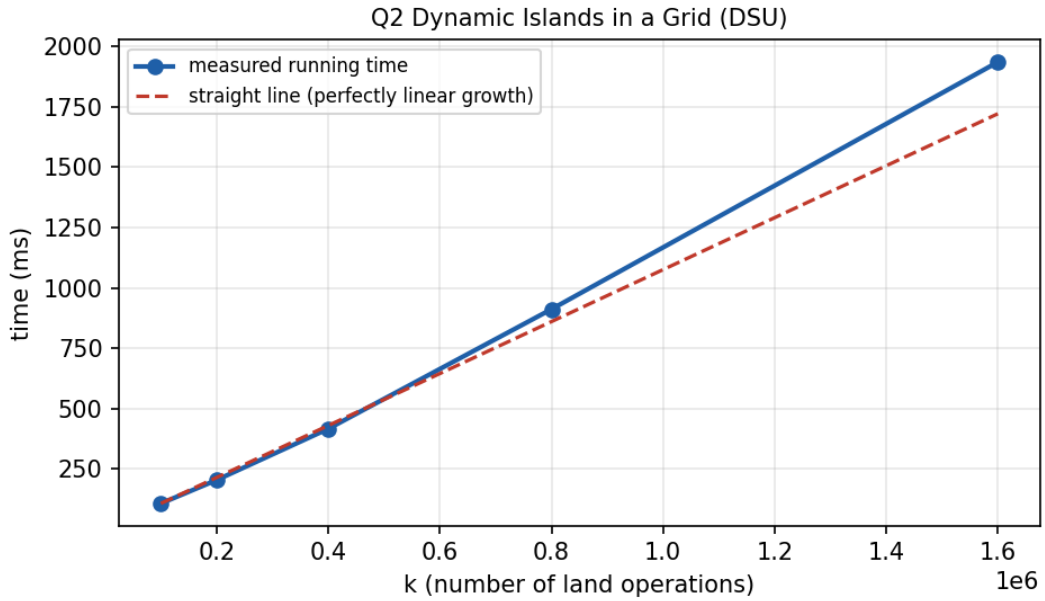
How it was derived

An operation touches at most four neighbours, so it performs at most 4 unions, i.e. at most 8 `find` calls plus constant work. With union by size and path halving each `find` is amortised `O(α(N))`. Hence `O(α(N))` per operation and `O(k α(N))` for all operations; the `O(N)` array initialisation is paid once. The naive method costs `O(k·N)`.

If the grid is huge but `k` is small, the flat array can be replaced by a hash map keyed by `id`, giving `O(k)` space.

14. Complexity Graph

The graph below shows the **measured running time** of the program (best of 3 runs, `gcc -O2`) for increasing input sizes (up to 1.6 million vertices or operations, with as many edges), together with a perfectly linear reference line. The measured times include reading the input with `scanf`, which is itself linear. The curve stays close to the straight line, which agrees with the near-linear bound.



Q2: measured running time against input size

15. Advantages

1. Each operation is answered in almost constant time.
2. The 1-D flattened array is simple, cache-friendly and holds both the grid and the DSU.
3. Duplicate additions and out-of-range positions are handled safely.
4. No recounting or graph search is needed.

16. Limitations

1. Only adding land is supported; removing land would need a different technique.
2. The flat array needs `O(R·C)` memory even if very few cells are used.
3. The printed final grid is limited to `30 × 30` to keep the output readable.

17. Viva Preparation

Q1. What is coordinate flattening?

Mapping a 2-D cell `(r, c)` to one integer `r*C + c`, so a 1-D array can store the whole grid and the DSU.

Q2. Why is DSU suitable here?

Land is only ever added, so islands only get created or merged, never split; these are exactly make-set and union operations.

Q3. How is the island count updated?

+1 for the new land cell, then −1 for every successful merge with a neighbour.

Q4. Why must `unite` return whether a merge happened?
Two neighbours may already belong to the same island; the counter must be decremented only for real merges.

Q5. What does `parent[id] == -1` mean?
The cell is water.

Q6. How are duplicate additions handled?
If `parent[id] != -1` the cell is already land, so nothing changes.

Q7. Why only 4 neighbours?
The problem defines connectivity as 4-directional.

Q8. What is the complexity?
`O(R·C + k α(R·C))` time and `O(R·C)` space.

Q9. What is path halving?
While walking up to the root, each node is pointed to its grandparent. It flattens the tree like path compression and needs only one pass.

Q10. What is the brute-force complexity?
`O(k·R·C)` by running BFS/DFS after every operation.

Q11. How would you reduce memory for a sparse grid?
Use a hash map from `id` to DSU node instead of an array of size `R·C`.

18. 1–2 Minute Teacher Explanation

> *"Land cells are added to an initially empty grid, and after each addition I must report the number of islands. Since land is never removed, islands can only appear or merge, which is exactly what a disjoint set union does.*
>
> *I flatten each cell (r, c) to id = r*C + c and keep one array where -1 means water and any other value is the DSU parent. When a cell becomes land I create a new set and increase the island counter. Then I look at the four neighbours; for each land neighbour I call unite, and if it really merged two different islands I decrease the counter.*
>
> *Because every operation needs at most four unions, each with amortised O(α(RC)) cost, the total time is O(RC + k α(RC)) and the space is O(RC). The naive approach of re-running BFS after each step would cost O(k·R·C).*"

19. How to Compile and Run

Save the program as `Q2.c`.

GCC (Linux / macOS)

```
gcc Q2.c -o Q2
./Q2
```

Windows

```
gcc Q2.c -o Q2.exe
Q2.exe
```

Using a file as input

```
./Q2 < input.txt > output.txt
```

20. Files in This Lab

1. `Q2.c` — C source code
2. `README.md` — complete explanation and lab documentation
3. `Q2_complexity.png` — measured running-time graph

21. Conclusion

Number of Islands II is a pure make-set / union workload: land is only added, so islands are created or merged but never split. Flattening coordinates to `r*C + c` lets one array represent both the grid and the

DSU, giving `O(R·C + k α(R·C))` time and `O(R·C)` space.

Q3. Maximum Weight Component with Size Constraint

1. Problem Title

Maximum Weight Component with Size Constraint

2. Problem Statement

A graph has `V` vertices, and vertex `v` has weight `wv`. Edges, each with a weight, are added **sequentially**. At any point we must report the maximum total vertex weight of any connected component whose **size** (number of vertices) does not exceed a threshold `K`.

Modelling assumptions

- The *weight of a component* is the sum of the weights of its **vertices**, as the statement says ("total vertex weight"). The edge weights are read but do not affect the answer.
- The *size* of a component is its number of vertices; a component is **valid** if its size is at most `K` (`K ≥ 1`, so every single vertex is initially valid).
- The answer is printed once before any edge and again after every edge.
- If every component is larger than `K`, the program prints that no valid component exists.
- Vertex weights may be any integers (negative weights are handled too).

3. Input Representation

Edges arrive online, so no adjacency structure is built. The state of the graph is stored in a DSU where each **root** also keeps aggregate data:

Array	Meaning
`parent[x]`	DSU parent
`size_[r]`	number of vertices in the component with root `r`
`sum[r]`	total vertex weight of the component with root `r` (`long long`)
`heap[]`	max-heap of candidate answers `(weight, root)`

```
V K E
w1 w2 ... wV
u1 v1 x1
u2 v2 x2
...
uE vE xE        (xi = edge weight, not used for the answer)
```

4. Possible Approaches

Approach 1: Recompute everything after every edge

Run BFS/DFS, find all components and take the maximum valid sum: `O(V + E)` per edge, `O(E(V+E))` overall.

Approach 2: DSU + scan of all roots

DSU maintains `size` and `sum` in near-constant time, but finding the maximum over all roots after each edge costs `O(V)`, giving `O(E · V)`.

Approach 3: DSU + max-heap with lazy deletion (used here)

The difficulty is that the answer can **decrease**: when the best component merges into another one and becomes larger than `K`, it disappears. A plain running maximum cannot handle that. A max-heap handles it if stale entries are discarded lazily when they reach the top.

5. Basic Idea

Components only ever **grow** when edges are added. Therefore a component that has become larger than `K` can never become valid again, and the only way a valid candidate disappears is by being merged into a bigger set.

- Whenever a merge creates a component of size `≤ K`, push `(sum, root)` into the max-heap.

- Old entries are never removed immediately. Instead, when answering, look at the top entry and test whether it is **still valid**:

```
entry (w, r) is valid  <=>  r is still a DSU root  AND  sum[r] == w  AND  size_[r] <= K
```

If the top is stale, pop it and look again. The first valid top is the maximum weight among all currently valid components, because every currently valid component has a (non-stale) entry in the heap.

6. Step-by-Step Algorithm

1. Read `V`, `K`, `E` and the vertex weights. Initialise each vertex as its own set: `size_ = 1`, `sum = weight`; push every `(weight, vertex)` into the heap (all are valid because `K ≥ 1`).
2. Report the answer for the initial graph.
3. For each edge `(u, v)`: find both roots; if they are equal, nothing changes.
4. Otherwise union by size, update `size_` and `sum` of the new root, and if the new size is `≤ K` push `(sum, root)` into the heap.
5. To report: while the heap is non-empty and its top is stale, pop it.
6. If the heap is empty, print that no component has size `≤ K`; otherwise print the top weight and the size of its component.

7. C Program

The complete program is in `Q3.c`. Prompts are written to `stderr` and results to `stdout`.

```
#include <stdio.h>
#include <stdlib.h>

/*
   Maximum weight component with size <= K, edges arriving online.

   DSU keeps for every root: size_[] and sum[] (total vertex weight).
   A max-heap with LAZY DELETION keeps candidate (sum, root) pairs.
   A heap entry is valid only if
        root is still a DSU root, sum[root] == entry.w, size_[root] <= K.
*/

typedef long long ll;

typedef struct { ll w; int root; } Entry;

int V, K, E;
int *parent, *size_;
ll *sum, *weight;

Entry *heap;
int heapSize = 0;

/* ---------- max-heap on Entry.w ---------- */
void heapPush(ll w, int root)
{
    int i = heapSize++;
    while (i > 0 && heap[(i - 1) / 2].w < w)
    {
        heap[i] = heap[(i - 1) / 2];
        i = (i - 1) / 2;
    }
    heap[i].w = w;
    heap[i].root = root;
}

void heapPop()
{
    Entry last = heap[--heapSize];
    int i = 0, child;
    while ((child = 2 * i + 1) < heapSize)
    {
        if (child + 1 < heapSize && heap[child + 1].w > heap[child].w)
            child++;
        if (heap[child].w <= last.w)
            break;
        heap[i] = heap[child];
        i = child;
    }
    if (heapSize > 0)
        heap[i] = last;
}
```

```
/* ---------- DSU ---------- */
int find(int x)
{
    while (parent[x] != x)
    {
        parent[x] = parent[parent[x]];
        x = parent[x];
    }
    return x;
}

int isValid(Entry e)
{
    return parent[e.root] == e.root && sum[e.root] == e.w
           && size_[e.root] <= K;
}

void addEdge(int u, int v)
{
    int ru = find(u), rv = find(v);
    if (ru == rv)
        return;
    if (size_[ru] < size_[rv])
    {
        int t = ru; ru = rv; rv = t;
    }
    parent[rv] = ru;
    size_[ru] += size_[rv];
    sum[ru] += sum[rv];
    if (size_[ru] <= K)          /* only a valid component is a candidate */
        heapPush(sum[ru], ru);
}

/* Prints the current answer; removes stale entries from the heap top */
void report(const char *label, int idx)
{
    while (heapSize > 0 && !isValid(heap[0]))
        heapPop();

    if (heapSize == 0)
        printf("%s %d: no component has size <= K\n", label, idx);
    else
        printf("%s %d: maximum weight = %lld (component size %d)\n",
               label, idx, heap[0].w, size_[heap[0].root]);
}

int main()
{
    int i, u, v;
    ll w;

    fprintf(stderr, "Enter V, K and E: ");
    if (scanf("%d %d %d", &V, &K, &E) != 3 || V <= 0 || K <= 0 || E < 0)
    {
        printf("V and K must be positive and E must not be negative.\n");
        return 0;
    }

    parent = (int *)malloc((V + 1) * sizeof(int));
    size_  = (int *)malloc((V + 1) * sizeof(int));
    sum    = (ll *)malloc((V + 1) * sizeof(ll));
    weight = (ll *)malloc((V + 1) * sizeof(ll));
    /* V initial entries + at most V-1 successful unions -> fewer than 2V entries */
    heap   = (Entry *)malloc((size_t)(2 * V + 2) * sizeof(Entry));
    if (!parent || !size_ || !sum || !weight || !heap)
    {
        printf("Memory allocation failed.\n");
        return 0;
    }

    fprintf(stderr, "Enter the %d vertex weights: ", V);
    for (i = 1; i <= V; i++)
    {
        if (scanf("%lld", &weight[i]) != 1)
        {
            printf("Invalid input.\n");
            return 0;
        }
        parent[i] = i;
        size_[i] = 1;
        sum[i] = weight[i];
        heapPush(sum[i], i);     /* every singleton is valid since K >= 1 */
    }

    report("Initially, before edge", 0);
```

```
    fprintf(stderr, "Enter the %d edges (u v edgeWeight):\n", E);
    for (i = 1; i <= E; i++)
    {
        fprintf(stderr, "Edge %d: ", i);
        if (scanf("%d %d %lld", &u, &v, &w) != 3 ||
            u < 1 || u > V || v < 1 || v > V)
        {
            printf("Invalid edge.\n");
            return 0;
        }
        /* The edge weight w does not change vertex weights, so it is not used. */
        addEdge(u, v);
        report("After edge", i);
    }

    free(parent); free(size_); free(sum); free(weight); free(heap);
    return 0;
}
```

8. Important Functions

`heapPush(w, root)` / `heapPop()`

A hand-written binary max-heap ordered by `w`. Both operations cost `O(log n)` for a heap of `n` entries.

`find(int x)`

Iterative DSU find with path halving.

`isValid(Entry e)`

Decides whether a heap entry still describes a real, currently valid component (still a root, same sum, size at most `K`).

`addEdge(int u, int v)`

Merges the components of `u` and `v` (if different), updates `size_` and `sum`, and pushes the new candidate if its size is at most `K`.

`report(label, idx)`

Discards stale entries from the top of the heap and prints the current maximum valid component weight.

9. Important Variables

Variable	Meaning
`V`, `K`, `E`	Number of vertices, size limit, number of edges
`weight[]`	Vertex weights
`parent[]`, `size_[]`	DSU parent and component size
`sum[]`	Total vertex weight of each component (valid at roots)
`heap[]`, `heapSize`	Max-heap of `(weight, root)` candidates
`Entry`	Structure `{ long long w; int root; }`

10. Sample Input

Type the values below (or redirect them from a file). The prompts are printed on the screen while you type; only the raw input is shown here.

```
6 3 5
5 1 4 7 2 3
1 2 10
3 4 20
2 3 5
5 6 7
1 5 2
```

11. Sample Output

```
Initially, before edge 0: maximum weight = 7 (component size 1)
After edge 1: maximum weight = 7 (component size 1)
After edge 2: maximum weight = 11 (component size 2)
After edge 3: maximum weight = 3 (component size 1)
After edge 4: maximum weight = 5 (component size 2)
After edge 5: no component has size <= K
```

This output was produced by compiling and running the program on the sample input above.

12. Dry Run

`V = 6`, `K = 3`, weights `[5, 1, 4, 7, 2, 3]`. Heap initially holds the six singletons.

Step	Components (weight)	Heap action	Answer
start	{1}=5 {2}=1 {3}=4 {4}=7 {5}=2 {6}=3	6 pushes	7
edge (1,2)	{1,2}=6 {3} {4}=7 {5} {6}	push 6; entries 5 and 1 become stale	7
edge (3,4)	{1,2}=6 {3,4}=11 {5} {6}	push 11	11
edge (2,3)	{1,2,3,4} has size 4 > K (invalid), {5}=2, {6}=3	no push; stale entries 11, 7, 6, 5, 4 are popped from the top	3
edge (5,6)	{1,2,3,4} (invalid) {5,6}=5	push 5	5
edge (1,5)	everything merged, size 6 > K	no push; all entries stale	no valid component

The answers printed by the program are `7, 7, 11, 3, 5, none` (the first 7 is before any edge). After edge 3 the answer **decreases** from 11 to 3, which is exactly the situation that requires the heap with lazy deletion.

13. Complexity Analysis

Heap entries are pushed only for the `V` initial vertices and for **successful** unions. There are at most `V − 1` successful unions, so the heap never holds more than `2V − 1` entries, and every entry is popped at most once.

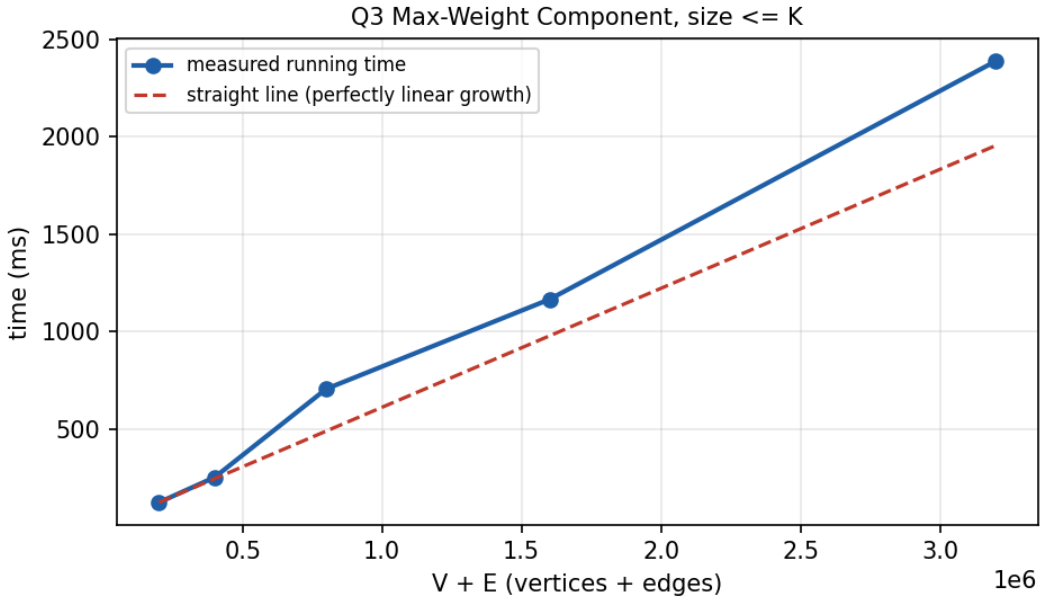
Quantity	Cost
Initialisation (V pushes)	`O(V log V)`
`E` edges: DSU work	`O(E · α(V))`
Pushes caused by merges (≤ V−1)	`O(V log V)`
Pops of stale entries (each entry popped at most once)	`O(V log V)` in total
Reading the top for each report	`O(1)` per edge
Total time	`O(E · α(V) + V log V)`
Space	`O(V)`

How it was derived

Each edge costs two `find` calls (`O(α(V))` amortised) and at most one push (`O(log V)`), but the number of pushes over the whole run is bounded by `2V − 1`, not by `E`. Stale entries are removed at most once each, so the total pop cost is also `O(V log V)`. Adding the parts gives `O(E α(V) + V log V)`. The simple alternatives cost `O(E · V)`.

14. Complexity Graph

The graph below shows the **measured running time** of the program (best of 3 runs, `gcc -O2`) for increasing input sizes (up to 1.6 million vertices or operations, with as many edges), together with a perfectly linear reference line. The measured times include reading the input with `scanf`, which is itself linear. The curve stays close to the straight line, which agrees with the near-linear bound.



Q3: measured running time against input size

15. Advantages

1. Correct even when the answer decreases (when the best component becomes too large).
2. Near-linear overall running time, much better than rescanning all components.
3. Works with negative vertex weights as well.
4. Uses only `O(V)` memory.

16. Limitations

1. Edges can only be added, not removed.
2. The edge weights are ignored because the objective depends only on vertex weights.
3. Weights are limited to the range of `long long`.
4. Component size is counted in vertices (not edges).

17. Viva Preparation

Q1. What does the DSU store in addition to parents?
For every root: the component size and the total vertex weight.

Q2. Why is a simple running maximum not enough?
A component that was the maximum can merge into a component larger than `K` and disappear, so the maximum can decrease.

Q3. What is lazy deletion?
Instead of deleting outdated heap entries immediately, we leave them and discard them when they reach the top and are found to be stale.

Q4. When is a heap entry `(w, r)` valid?
When `r` is still a root, `sum[r] == w` and `size_[r] <= K`.

Q5. Why can a component never become valid again once it is larger than `K`?
Components only grow when edges are added.

Q6. Why do we push only if the new size is at most `K`?
A component larger than `K` can never be an answer, so it is useless in the heap.

Q7. Why is the heap size at most `2V − 1`?
V initial pushes plus at most V−1 successful unions.

Q8. What is the total complexity?
`O(E α(V) + V log V)` time and `O(V)` space.

Q9. What happens if the edge joins two vertices already in the same component?
Nothing changes, so the previous answer remains valid.

Q10. What is the role of the edge weights?
None for this objective; they are read and ignored because only vertex weights are summed.

Q11. Why use `long long`?
Sums of many large vertex weights can overflow `int`.

18. 1–2 Minute Teacher Explanation

> *"I use a DSU in which every root stores the component size and total vertex weight. When an edge connects two different components I merge them and add their sizes and weights.*
>
> *The tricky part is the query, because the answer can go down: if the best component merges into one that becomes larger than K, it is no longer allowed. So I keep a max-heap of candidate (weight, root) pairs and I push a new pair whenever a merge produces a component of size at most K. I never delete old pairs immediately; at query time I look at the top and check that its root is still a root, that its stored weight equals the current weight and that the size is still at most K. If not, I discard it and look at the next one.*
>
> *There are at most 2V entries because pushes come only from the V initial vertices and at most V-1 merges, and each entry is popped at most once. So the total time is O(E α(V) + V log V) and the space is O(V).*"

19. How to Compile and Run

Save the program as `Q3.c`.

GCC (Linux / macOS)

```
gcc Q3.c -o Q3
./Q3
```

Windows

```
gcc Q3.c -o Q3.exe
Q3.exe
```

Using a file as input

```
./Q3 < input.txt > output.txt
```

20. Files in This Lab

1. `Q3.c` — C source code
2. `README.md` — complete explanation and lab documentation
3. `Q3_complexity.png` — measured running-time graph

21. Conclusion

The only subtle point is that the best valid component can disappear when it merges into a component larger than `K`. A DSU that maintains size and weight, combined with a max-heap using lazy deletion, handles this and gives `O(E α(V) + V log V)` time and `O(V)` space.

Q4. Reverse Time-Step DSU: Maximum Component Sum under Vertex Deletions

1. Problem Title

Reverse Time-Step DSU: Maximum Component Sum under Vertex Deletions

2. Problem Statement

A vertex-weighted undirected graph is given. Vertices are deleted one by one in the order `P = [p1, p2, …, pV]`. Deleting a vertex removes it **and all its incident edges**. After each deletion we must output the maximum sum of vertex values among all remaining connected components.

Modelling assumptions

- Vertices are numbered `1..V` and `P` is a permutation of `1..V`.
- Vertex values are **non-negative** integers (the standard assumption for this problem; it makes the component sum behave monotonically when components are merged).
- After the last deletion the graph is empty, so the answer is `0`.
- Self-loops and parallel edges are allowed and cause no problems.
- The program additionally prints the answer *before* any deletion.

3. Input Representation

This problem needs to know the neighbours of a vertex at the moment that vertex is (re)inserted, so the graph is stored as an **adjacency list implemented with arrays** (forward-star / linked-list-in-arrays), which uses `O(V + E)` memory and no pointer allocation:

Array	Meaning
`head[v]`	index of the first edge of vertex `v` (`-1` if none)
`to[e]`, `nxt[e]`	the other end of edge `e` and the next edge of the same vertex
`weight[v]`	value of vertex `v`
`P[i]`	the i-th vertex to be deleted
`active[v]`	1 if `v` is currently present in the reverse process
`parent`, `size_`, `sum`	DSU arrays; `sum[r]` is the total value of the component of root `r`

```
V E
w1 w2 ... wV
u1 v1
...
uE vE
p1 p2 ... pV
```

4. Possible Approaches

Approach 1: Recompute after every deletion

Delete the vertex, run BFS/DFS and take the maximum component sum: `O(V + E)` per deletion, `O(V(V + E))` in total.

Approach 2: Forward DSU

DSU supports only *merging*; it cannot split a set when a vertex is removed, so it cannot be applied directly to deletions.

Approach 3: Reverse time with DSU (used here)

Run the process **backwards**: deletions become insertions, and insertions only merge components, which DSU does perfectly.

5. Basic Idea

Let `ans[i]` be the answer after deleting `p1, …, pi`. This is the best component sum of the graph induced by the vertices `p(i+1), …, pV`. So if we start with an **empty graph** and insert `pV`, then `p(V−1)`, ..., then `p1`, the graph just before inserting `p_i` is exactly the graph remaining after the i-th deletion.

Insertion of a vertex `x`:

1. create the set `{x}` with sum `weight[x]`;
2. for every neighbour `u` of `x` that is already present, merge the components of `x` and `u`;
3. maintain `best`, the maximum component sum so far.

Because vertex values are non-negative, merging never decreases a component's sum, so a single running maximum `best` is enough (in the reverse process components only grow).

```
for i = V down to 0:
    ans[i] = best            // graph = {p(i+1), ..., pV}
    if i > 0: insert p_i and update best
```

6. Step-by-Step Algorithm

1. Read the graph, the vertex values and the deletion order `P`; build the adjacency lists (each edge stored in both directions).
2. Set `best = 0` and `active[·] = 0`.
3. For `i = V` down to `0`: set `ans[i] = best`.
4. If `i > 0`, insert `p_i`: make it a singleton set, update `best` with its weight.
5. For each neighbour `u` of `p_i` with `active[u] = 1`: if `find(p_i) != find(u)` merge them, add the sums and update `best`.
6. Print `ans[0]` (before any deletion) and then `ans[1..V]` in deletion order.

7. C Program

The complete program is in `Q4.c`. Prompts are written to `stderr` and results to `stdout`.

```
#include <stdio.h>
#include <stdlib.h>

/*
   Reverse time-step DSU.
   Deleting vertices p1..pV one by one is hard for DSU (it cannot split sets),
   so we run time backwards: start with an empty graph and ADD
   pV, pV-1, ..., p1 and record the best component sum before each addition.
*/

typedef long long ll;

int V, E;
int *head, *nxt, *to;          /* adjacency list stored in arrays */
int *parent, *size_, *active;
ll *weight, *sum, *ans;
int *P;

int find(int x)
{
    while (parent[x] != x)
    {
        parent[x] = parent[parent[x]];
        x = parent[x];
    }
    return x;
}

void addEdgeToList(int a, int b, int *cnt)
{
    to[*cnt] = b;
    nxt[*cnt] = head[a];
    head[a] = (*cnt)++;
}

int main()
{
    int i, a, b, cnt = 0, e;
    ll best = 0;

    fprintf(stderr, "Enter number of vertices V and edges E: ");
    if (scanf("%d %d", &V, &E) != 2 || V <= 0 || E < 0)
    {
```

```
    printf("V must be positive and E must not be negative.\n");
    return 0;
}

head   = (int *)malloc((V + 1) * sizeof(int));
nxt    = (int *)malloc((size_t)(2 * E + 1) * sizeof(int));
to     = (int *)malloc((size_t)(2 * E + 1) * sizeof(int));
parent = (int *)malloc((V + 1) * sizeof(int));
size_  = (int *)malloc((V + 1) * sizeof(int));
active = (int *)calloc(V + 1, sizeof(int));
weight = (ll *)malloc((V + 1) * sizeof(ll));
sum    = (ll *)malloc((V + 1) * sizeof(ll));
ans    = (ll *)malloc((V + 2) * sizeof(ll));
P      = (int *)malloc((V + 1) * sizeof(int));
if (!head || !nxt || !to || !parent || !size_ || !active ||
    !weight || !sum || !ans || !P)
{
    printf("Memory allocation failed.\n");
    return 0;
}
for (i = 1; i <= V; i++)
    head[i] = -1;

fprintf(stderr, "Enter the %d vertex values (non-negative): ", V);
for (i = 1; i <= V; i++)
{
    if (scanf("%lld", &weight[i]) != 1 || weight[i] < 0)
    {
        printf("Vertex values must be non-negative integers.\n");
        return 0;
    }
}

fprintf(stderr, "Enter the %d edges (u v):\n", E);
for (i = 1; i <= E; i++)
{
    if (scanf("%d %d", &a, &b) != 2 || a < 1 || a > V || b < 1 || b > V)
    {
        printf("Invalid edge.\n");
        return 0;
    }
    addEdgeToList(a, b, &cnt);
    addEdgeToList(b, a, &cnt);
}

fprintf(stderr, "Enter the deletion order P (a permutation of 1..%d): ", V);
for (i = 1; i <= V; i++)
{
    if (scanf("%d", &P[i]) != 1 || P[i] < 1 || P[i] > V || active[P[i]])
    {
        printf("P must be a permutation of 1..%d.\n", V);
        return 0;
    }
    active[P[i]] = 1;        /* temporarily used as a "seen" flag */
}
for (i = 1; i <= V; i++)
    active[i] = 0;

/* ---- reverse process ---- */
for (i = V; i >= 0; i--)
{
    ans[i] = best;           /* best sum after deleting p1..pi */
    if (i == 0)
        break;

    a = P[i];                /* re-insert vertex p_i */
    active[a] = 1;
    parent[a] = a;
    size_[a] = 1;
    sum[a] = weight[a];
    if (sum[a] > best)
        best = sum[a];

    for (e = head[a]; e != -1; e = nxt[e])
    {
        int u = to[e];
        int ra, rb;
        if (!active[u])
            continue;
        ra = find(a);
        rb = find(u);
        if (ra == rb)
            continue;
        if (size_[ra] < size_[rb])
        {
            int t = ra; ra = rb; rb = t;
```

```
            }
            parent[rb] = ra;
            size_[ra] += size_[rb];
            sum[ra] += sum[rb];
            if (sum[ra] > best)
                best = sum[ra];
        }
    }

    printf("Before any deletion: maximum component sum = %lld\n", ans[0]);
    for (i = 1; i <= V; i++)
        printf("After deleting vertex %d: maximum component sum = %lld\n",
               P[i], ans[i]);

    return 0;
}
```

8. Important Functions

`addEdgeToList(a, b, &cnt)`

Inserts the directed edge `a → b` at the front of the list of `a` (called twice per undirected edge).

`find(int x)`

Iterative DSU find with path halving.

`main()`

Reads and validates the input (including that `P` is a permutation), runs the reverse insertion loop with DSU merges, and prints the answers in the original deletion order.

9. Important Variables

Variable	Meaning
`V`, `E`	Number of vertices and edges
`weight[]`	Vertex values
`head[]`, `nxt[]`, `to[]`	Adjacency list stored in arrays
`P[]`	Deletion order
`active[]`	Whether a vertex has been re-inserted
`parent[]`, `size_[]`, `sum[]`	DSU arrays with component sums
`best`	Maximum component sum currently present
`ans[]`	`ans[i]` = answer after i deletions

10. Sample Input

Type the values below (or redirect them from a file). The prompts are printed on the screen while you type; only the raw input is shown here.

```
5 4
4 2 7 3 5
1 2
2 3
3 4
4 5
3 1 5 2 4
```

11. Sample Output

```
Before any deletion: maximum component sum = 21
After deleting vertex 3: maximum component sum = 8
After deleting vertex 1: maximum component sum = 8
After deleting vertex 5: maximum component sum = 3
After deleting vertex 2: maximum component sum = 3
After deleting vertex 4: maximum component sum = 0
```

This output was produced by compiling and running the program on the sample input above.

12. Dry Run

Path graph `1 − 2 − 3 − 4 − 5` with values `[4, 2, 7, 3, 5]` (total 21) and deletion order `P = [3, 1, 5, 2, 4]`. We process the order in reverse.

i	`ans[i]` stored (current `best`)	Insert `p_i`	Components now	`best`
5	0	4 (value 3)	{4}=3	3
4	3	2 (value 2)	{4}=3, {2}=2	3
3	3	5 (value 5), merges with 4	{4,5}=8, {2}=2	8
2	8	1 (value 4), merges with 2	{1,2}=6, {4,5}=8	8
1	8	3 (value 7), merges with 2 and 4	{1,2,3,4,5}=21	21
0	21	—	—	—

Reading `ans[1..5]` forwards: `8, 8, 3, 3, 0`, with `ans[0] = 21` before any deletion. Check by hand: deleting vertex 3 leaves `{1,2} = 6` and `{4,5} = 8`, so the answer is 8. ✓

13. Complexity Analysis

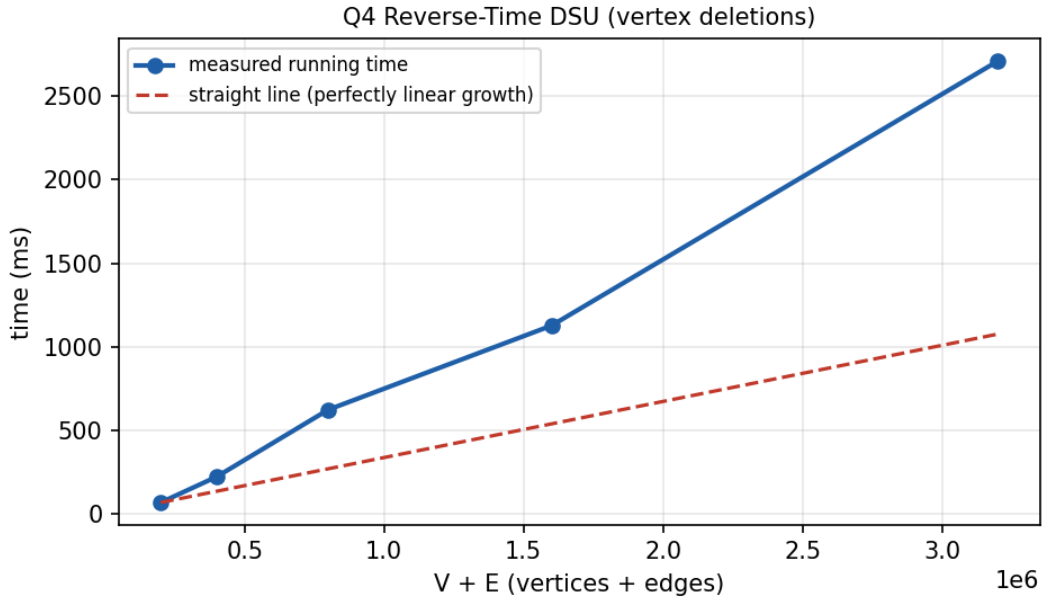
Quantity	Cost
Building the adjacency lists	`O(V + E)`
Reverse loop: every vertex is inserted once	`O(V)`
Every edge is examined exactly twice (once from each end), each time with ≤ 2 `find` calls	`O(E · α(V))`
Total time	`O(V + E · α(V))`, practically linear
Space	`O(V + E)`

How it was derived

In the reverse process each vertex is activated once, and while activating it we walk through its adjacency list. Summed over all vertices, the lists have total length `2E`. For each entry we do at most two `find` calls, each amortised `O(α(V))` with union by size and path halving. Therefore the whole run takes `O(V + E α(V))`. The naive recomputation costs `O(V(V + E))`.

14. Complexity Graph

The graph below shows the **measured running time** of the program (best of 3 runs, `gcc -O2`) for increasing input sizes (up to 1.6 million vertices or operations, with as many edges), together with a perfectly linear reference line. The measured times include reading the input with `scanf`, which is itself linear. The curve stays close to the straight line, which agrees with the near-linear bound.



Q4: measured running time against input size

15. Advantages

1. Converts a hard deletion problem into an easy insertion problem.
2. Near-linear time: every vertex and edge is handled a constant number of times.
3. A single `best` variable is enough because components only grow in reverse.
4. Handles self-loops and repeated edges without special code.

16. Limitations

1. The whole deletion order must be known in advance (the algorithm is offline).
2. It relies on non-negative vertex values; with negative values a single running maximum is not sufficient.
3. The adjacency lists need `O(E)` memory, unlike the online problems.

17. Viva Preparation

Q1. Why can't DSU be used directly for deletions?
DSU supports union but not splitting a set, and deleting a vertex may split a component.

Q2. What is the trick?
Reverse time: add the vertices in the reverse order of deletion so that every step is a merge.

Q3. What does `ans[i]` represent?
The maximum component sum after the first `i` deletions, i.e. in the graph made of `p(i+1)..pV`.

Q4. Why is `ans[V] = 0`?
After deleting all vertices the graph is empty.

Q5. Why is one `best` variable enough?
In the reverse process components only grow and values are non-negative, so sums never decrease.

Q6. Why must vertex values be non-negative?
With negative values merging could reduce a sum, and the maximum would no longer be monotone.

Q7. Why store each edge twice?
The graph is undirected; each vertex needs to see its neighbours when it is inserted.

Q8. Why check `active[u]`?
Only neighbours that have already been re-inserted exist in the current graph.

Q9. What is the time complexity?
`O(V + E α(V))` with `O(V + E)` space.

Q10. What is the complexity of the naive method?

`O(V(V + E))`.

Q11. Is this algorithm online?

No, it is offline: it needs the complete deletion order beforehand.

18. 1–2 Minute Teacher Explanation

> *"Deleting vertices splits components, but a DSU can only merge. So I reverse time. The graph that remains after the i-th deletion is made of the vertices p(i+1) to pV, so if I start from an empty graph and add pV, then p(V-1), and so on, then just before adding p_i the graph is exactly what remains after the i-th deletion. I store the current best component sum at that moment as the answer for step i.*
>
> *To add a vertex I make a new set with its value and merge it with every neighbour that has already been added; each component keeps its total value in the DSU root and I keep one running maximum. Since values are non-negative and components only grow in reverse, the running maximum is correct. Finally I print the stored answers in forward order.*
>
> *Every vertex is added once and every edge is looked at twice, so the time is O(V + E α(V)) and the space is O(V + E)."*

19. How to Compile and Run

Save the program as `Q4.c`.

GCC (Linux / macOS)

```
gcc Q4.c -o Q4
./Q4
```

Windows

```
gcc Q4.c -o Q4.exe
Q4.exe
```

Using a file as input

```
./Q4 < input.txt > output.txt
```

20. Files in This Lab

1. `Q4.c` — C source code
2. `README.md` — complete explanation and lab documentation
3. `Q4_complexity.png` — measured running-time graph

21. Conclusion

Vertex deletion splits components, which DSU cannot do, so the problem is solved backwards: vertices are inserted in reverse order and the best component sum is recorded before each insertion. The time is `O(V + E α(V))` and the space is `O(V + E)`.

Q5. Lexicographically Smallest Permutation using Allowed Swaps

1. Problem Title

Lexicographically Smallest Permutation using Allowed Swaps

2. Problem Statement

A permutation `A` of the numbers `1..n` is given, together with a list of allowed pairwise swaps `(u, v)` that form the edges of an undirected graph. Any number of the allowed swaps may be performed, in any order, any number of times. Find the **lexicographically smallest** permutation that can be obtained.

Modelling assumptions

- A swap `(u, v)` exchanges the elements at **positions** `u` and `v` (positions are `1..n`).
- Swaps may be used repeatedly and in any order.
- `A` is validated to be a true permutation of `1..n`.

3. Input Representation

The graph is on the **positions** `1..n`; its edges are the allowed swaps. The graph itself never has to be stored: it is enough to feed each swap to a DSU. The program stores only arrays of size `n + 2`:

Array	Meaning
`A[i]`	value at position `i`
`posOf[v]`	position where value `v` is located
`parent`, `size_`	DSU over positions
`start`, `grp`	positions of every component laid out contiguously in increasing order (CSR layout)
`res[i]`	final answer

```
n
A1 A2 ... An
m
u1 v1
...
um vm
```

4. Possible Approaches

Approach 1: Greedy swapping

Repeatedly swap any pair where the left element is larger. This can get stuck and is not guaranteed to reach the smallest arrangement, because a useful move may need a chain of swaps through other positions.

Approach 2: DSU + sorting each component

Find components, collect values and positions of each component, sort both, and assign. Correct, with `O(n log n)` for sorting.

Approach 3: DSU + counting by values (used here)

Use the same component idea but avoid sorting: go through the values `1, 2, …, n` in increasing order and give each value to the smallest still-free position of its component. Because `A` is a permutation, the values are already available in sorted order, so the running time becomes (almost) linear.

5. Basic Idea

Key fact: if positions are connected by allowed swaps, then the values sitting in these positions can be rearranged in **any** order (any permutation of a connected set of positions can be produced by swaps along the edges of a connected graph, e.g. along a spanning tree). Therefore:

1 positions in different components never exchange values;

2 inside one component the values can be arranged freely.

To obtain the lexicographically smallest result, inside every component place the **smallest values at the smallest positions**. Since each component can be optimised independently and the smallest position overall always gets the smallest value its component owns, this is globally optimal.

6. Step-by-Step Algorithm

1 Read `n` and `A`; record `posOf[A[i]] = i`; make every position a DSU singleton.

2 For every allowed swap `(u, v)` do `unite(u, v)`.

3 Group positions by root. Counting positions per root and prefix-summing gives the start of each group in `grp[]`; scanning positions `i = 1..n` in order and appending `i` to the group of `find(i)` keeps every group in **increasing** position order.

4 For `v = 1, 2, …, n` in this order: `r = find(posOf[v])`; place `v` at the next unused slot of group `r` (`res[grp[ptr[r]++]] = v`).

5 Print `res[1..n]`.

7. C Program

The complete program is in `Q5.c`. Prompts are written to `stderr` and results to `stdout`.

```
#include <stdio.h>
#include <stdlib.h>

/*
   Lexicographically smallest permutation using allowed swaps.
   Positions that are connected by swaps form a component, and inside a
   component the values can be arranged in ANY order. So the smallest
   values of a component must go to its smallest positions.
*/

int n, m;
int *parent, *size_;

int find(int x)
{
    while (parent[x] != x)
    {
        parent[x] = parent[parent[x]];
        x = parent[x];
    }
    return x;
}

void unite(int a, int b)
{
    int ra = find(a), rb = find(b);
    if (ra == rb)
        return;
    if (size_[ra] < size_[rb])
    {
        int t = ra; ra = rb; rb = t;
    }
    parent[rb] = ra;
    size_[ra] += size_[rb];
}

int main()
{
    int i, u, v;
    int *A, *posOf, *start, *ptr, *grp, *res;

    fprintf(stderr, "Enter n: ");
    if (scanf("%d", &n) != 1 || n <= 0)
    {
        printf("n must be positive.\n");
        return 0;
    }

    A      = (int *)malloc((n + 2) * sizeof(int));
    posOf  = (int *)calloc(n + 2, sizeof(int));
    parent = (int *)malloc((n + 2) * sizeof(int));
    size_  = (int *)malloc((n + 2) * sizeof(int));
    start  = (int *)calloc(n + 2, sizeof(int));
    ptr    = (int *)calloc(n + 2, sizeof(int));
    grp    = (int *)malloc((n + 2) * sizeof(int));
    res    = (int *)malloc((n + 2) * sizeof(int));
```

```
    if (!A || !posOf || !parent || !size_ || !start || !ptr || !grp || !res)
    {
        printf("Memory allocation failed.\n");
        return 0;
    }

    fprintf(stderr, "Enter the permutation A[1..%d]: ", n);
    for (i = 1; i <= n; i++)
    {
        if (scanf("%d", &A[i]) != 1 || A[i] < 1 || A[i] > n || posOf[A[i]])
        {
            printf("A must be a permutation of 1..%d.\n", n);
            return 0;
        }
        posOf[A[i]] = i;
        parent[i] = i;
        size_[i] = 1;
    }

    fprintf(stderr, "Enter number of allowed swaps m: ");
    if (scanf("%d", &m) != 1 || m < 0)
    {
        printf("m must not be negative.\n");
        return 0;
    }

    fprintf(stderr, "Enter the %d swaps (u v), positions are 1..%d:\n", m, n);
    for (i = 1; i <= m; i++)
    {
        if (scanf("%d %d", &u, &v) != 2 || u < 1 || u > n || v < 1 || v > n)
        {
            printf("Invalid swap.\n");
            return 0;
        }
        unite(u, v);
    }

    /* Group the positions of every component in increasing order (CSR layout):
       start[r] = first slot of root r inside grp[] */
    for (i = 1; i <= n; i++)
        start[find(i) + 1]++;
    for (i = 1; i <= n + 1; i++)
        start[i] += start[i - 1];
    for (i = 1; i <= n; i++)
        ptr[i] = start[i];
    for (i = 1; i <= n; i++)
    {
        int r = find(i);
        grp[ptr[r]++] = i;       /* positions arrive in increasing order */
    }
    for (i = 1; i <= n; i++)
        ptr[i] = start[i];

    /* Give values 1,2,3,... to the smallest free position of their component */
    for (v = 1; v <= n; v++)
    {
        int r = find(posOf[v]);
        res[grp[ptr[r]++]] = v;
    }

    printf("Lexicographically smallest permutation:\n");
    for (i = 1; i <= n; i++)
        printf("%d%c", res[i], i == n ? '\n' : ' ');

    return 0;
}
```

8. Important Functions

`find(int x)` **and** `unite(int a, int b)`

Standard DSU operations: iterative `find` with path halving, and `unite` by size.

`main()`

Reads and validates input, unites positions, builds the contiguous groups, then fills `res[]` by giving every value, in increasing order, the smallest free position of its component. `ptr[r]` is the next free slot in group `r`.

9. Important Variables

Variable	Meaning
`n`, `m`	Permutation length and number of allowed swaps
`A[]`	Input permutation
`posOf[]`	Position of each value
`parent[]`, `size_[]`	DSU over positions
`start[]`	Offset of each component's group in `grp[]`
`grp[]`	Positions of all components, grouped, increasing
`ptr[]`	Next free slot in each group
`res[]`	Resulting smallest permutation

10. Sample Input

Type the values below (or redirect them from a file). The prompts are printed on the screen while you type; only the raw input is shown here.

```
6
6 3 5 1 4 2
3
1 2
2 4
3 5
```

11. Sample Output

```
Lexicographically smallest permutation:
1 3 4 6 5 2
```

This output was produced by compiling and running the program on the sample input above.

12. Dry Run

`n = 6`, `A = [6, 3, 5, 1, 4, 2]`, swaps `(1,2)`, `(2,4)`, `(3,5)`.

Component (positions)	Values currently there	Positions sorted	Values sorted	Result
{1, 2, 4}	6, 3, 1	1, 2, 4	1, 3, 6	pos 1 = 1, pos 2 = 3, pos 4 = 6
{3, 5}	5, 4	3, 5	4, 5	pos 3 = 4, pos 5 = 5
{6}	2	6	2	pos 6 = 2

Final permutation: `1 3 4 6 5 2`. The program obtains the same result without sorting: value 1 is at position 4 (component {1,2,4}) and gets the smallest free position 1; value 2 (component {6}) gets position 6; value 3 gets position 2; value 4 (component {3,5}) gets position 3; value 5 gets position 5; value 6 gets position 4.

13. Complexity Analysis

Quantity	Cost
Reading `A` and initialising	`O(n)`
`m` unions	`O(m · α(n))`
Grouping positions by root (counting sort / CSR)	`O(n · α(n))`
Assigning values `1..n` to positions	`O(n · α(n))`
Total time	`O(n + m · α(n))`, practically linear
Space	`O(n)`

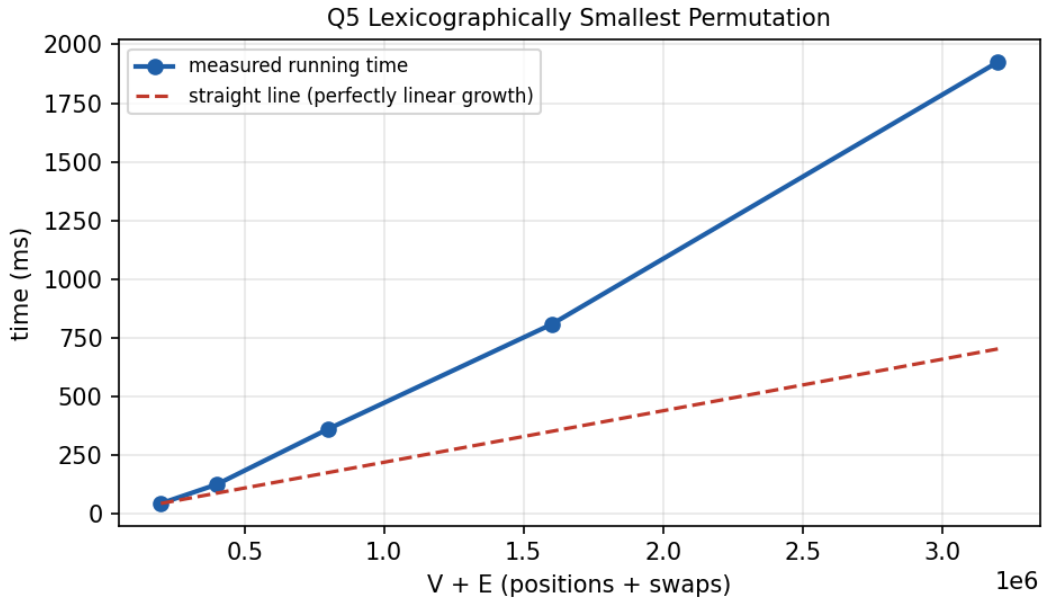
How it was derived

The DSU phase processes `m` edges at amortised `O(α(n))` each. The grouping and assignment phases each touch every position (or value) a constant number of times, with one `find` per touch. No sorting is needed because values are visited in increasing order `1..n` and positions are collected in increasing order by a

single left-to-right scan. If values were not a permutation, sorting inside every component would add `O(n log n)`.

14. Complexity Graph

The graph below shows the **measured running time** of the program (best of 3 runs, `gcc -O2`) for increasing input sizes (up to 1.6 million vertices or operations, with as many edges), together with a perfectly linear reference line. The measured times include reading the input with `scanf`, which is itself linear. The curve stays close to the straight line, which agrees with the near-linear bound.



Q5: measured running time against input size

15. Advantages

1. Near-linear time; no sorting step is required.
2. Uses only `O(n)` memory and never stores the swap graph.
3. Provably optimal: components are independent and each is arranged in increasing order.
4. Input validation guarantees `A` really is a permutation.

16. Limitations

1. Assumes that swaps exchange *positions*; a swap on *values* would need the graph to be built over values instead.
2. The values must form a permutation of `1..n` for the counting-by-value trick; for arbitrary values each component must be sorted.

17. Viva Preparation

Q1. Why do connected positions allow any arrangement?

Adjacent transpositions along a spanning tree of a connected graph generate every permutation of its vertices.

Q2. Why is DSU used?

It finds the connected components of the swap graph in almost linear time.

Q3. What is the greedy rule inside a component?

Put the smallest values at the smallest positions.

Q4. Why is this lexicographically smallest overall?

Components are independent, and in each one the earliest position gets the smallest value that component can supply.

Q5. Why is sorting not needed in this program?
Values are processed in increasing order `1..n` and positions are collected in increasing order, so both lists are already sorted.

Q6. What does `ptr[r]` do?
It points to the next free (smallest unused) position in the group of root `r`.

Q7. What is CSR layout?
Storing all groups one after another in one array with a `start[]` offset for each group.

Q8. What is the complexity?
`O(n + m α(n))` time, `O(n)` space.

Q9. What if no swap is allowed?
Every position is its own component, so the permutation is unchanged.

Q10. What if the swaps are on values rather than positions?
Build the DSU over values; the components are then groups of values that can exchange their places.

Q11. Why can't simple greedy swapping be trusted?
It may stop at a local state; the optimal arrangement may need swaps through intermediate positions.

18. 1–2 Minute Teacher Explanation

> *"If two positions are connected by allowed swaps, then the elements in the whole connected group can be rearranged in any order. So I treat positions as vertices, allowed swaps as edges, and use a DSU to find the connected components.*
>
> *Inside each component I want the smallest values at the smallest positions. I do this without sorting: I group the positions of every component in increasing order, and then I go through the values 1, 2, 3, … n. For each value I find the component that currently contains it and put the value into the smallest unused position of that component.*
>
> *Components do not interact and each is filled optimally, so the result is the lexicographically smallest permutation. The DSU costs O(m α(n)) and the filling costs O(n α(n)), so the total is O(n + m α(n)) time and O(n) space."*

19. How to Compile and Run

Save the program as `Q5.c`.

GCC (Linux / macOS)

```
gcc Q5.c -o Q5
./Q5
```

Windows

```
gcc Q5.c -o Q5.exe
Q5.exe
```

Using a file as input

```
./Q5 < input.txt > output.txt
```

20. Files in This Lab

1. `Q5.c` — C source code
2. `README.md` — complete explanation and lab documentation
3. `Q5_complexity.png` — measured running-time graph

21. Conclusion

Positions connected by swaps form components whose values can be rearranged freely, so each component should hold its smallest values at its smallest positions. DSU finds the components and a counting-style assignment avoids sorting, giving `O(n + m α(n))` time and `O(n)` space.